Historical Manuscript Fragment Reconstruction & Evidence Explorer

Software Requirements and Architecture

Sai Ruthvik Reddy Aella

FA26 - COMP-699-A - PROFESSIONAL SEMINAR

Professor. David Pitts

October 18, 2026

1. Problem Definition and Project Scope

I plan to develop a Python application that helps researchers organize incomplete or separated historical manuscript pages. Comparing many pages manually can take a lot of time, especially when page numbers, text, or catalogue information are missing.

My application will compare page images, text, keywords, layout, and available metadata to suggest possible page sequences. It will show the evidence behind each suggestion so users can review the results and make their own decisions.

My main goal is to reduce the time needed to compare manuscript pages. I will use publicly available manuscript data and test the application with pages whose original order is known.

The project will include image and text comparison, metadata processing, candidate ranking, ordering constraints, local storage, and a simple user interface. I will not attempt to recreate missing historical content, guarantee the original order, or replace expert judgment. I will also exclude advanced handwriting recognition and large-scale archival management to keep the project realistic for one semester.

2. Users and Stakeholders

My main users will be archivists, librarians, museum staff, and history researchers. They will use the application to compare pages, review candidate sequences, and understand the evidence supporting each match.

Students and other interested users may also use it to explore manuscript pages. I will keep the interface simple so users do not need programming knowledge.

Libraries and digital archives will be important stakeholders because their public collections may provide the images and metadata I need. As the developer, I will be responsible for implementation, testing, data selection, and documentation.

3. Functional Requirements

I will use the following functional requirements to guide implementation and testing.

Input and Metadata

- **FR-01:** I will allow users to upload supported manuscript image files.
- **FR-02:** I will allow users to enter manuscript titles and source information.
- **FR-03:** I will allow users to enter page numbers when available.
- **FR-04:** I will allow users to enter or import page text.
- **FR-05:** I will allow users to enter available catalogue metadata.
- **FR-06:** I will validate uploaded file formats.
- **FR-07:** I will identify missing metadata fields.
- **FR-08:** I will allow users to view uploaded page records.
- **FR-09:** I will allow users to remove incorrect page records.
- **FR-10:** I will associate each page with its manuscript record.

Text and Image Processing

- **FR-11:** I will extract basic image properties, including dimensions.
- **FR-12:** I will prepare images for comparison using image-processing methods.
- **FR-13:** I will compare image layout and visual characteristics.
- **FR-14:** I will calculate image similarity scores.
- **FR-15:** I will compare page text using approximate text matching.
- **FR-16:** I will identify shared keywords between pages.
- **FR-17:** I will calculate text similarity scores when text is available.
- **FR-18:** I will compare available page numbers.
- **FR-19:** I will compare relevant manuscript metadata.
- **FR-20:** I will handle pages with missing text or metadata.

Matching and Ordering

- **FR-21:** I will combine available evidence into a candidate matching score.
- **FR-22:** I will allow the matching process to use configurable weights.
- **FR-23:** I will create a graph representing possible page relationships.
- **FR-24:** I will rank candidate page matches by their scores.
- **FR-25:** I will apply known ordering constraints.
- **FR-26:** I will remove candidate sequences that violate defined constraints.
- **FR-27:** I will generate possible page sequences from candidate relationships.
- **FR-28:** I will rank valid sequences using their available evidence.
- **FR-29:** I will identify cases where the evidence is insufficient.
- **FR-30:** I will display the evidence used for each proposed match.

Results and Storage

- **FR-31:** I will display ranked candidate page matches.
- **FR-32:** I will display proposed page sequences.
- **FR-33:** I will allow users to inspect the images of candidate pages.
- **FR-34:** I will display the individual evidence scores where available.
- **FR-35:** I will allow users to compare alternative sequences.
- **FR-36:** I will save manuscript records in a local database.
- **FR-37:** I will save candidate matches and analysis results.
- **FR-38:** I will allow users to run the analysis again after changing the input.
- **FR-39:** I will provide an evaluation view comparing proposed sequences with known orders.

- **FR-40:** I will display understandable error messages when processing fails.

4. Nonfunctional Requirements

I will use the following requirements to define the expected quality of the application.

Performance

- **NFR-01:** I will aim to process 50 test page images and display results within 30 seconds on my development laptop.
- **NFR-02:** I will reduce unnecessary image processing by reusing extracted features when possible.

Usability and Accessibility

- **NFR-03:** I will use clear labels and simple navigation.
- **NFR-04:** I will display errors and missing information in understandable language.
- **NFR-05:** I will make the main workflow usable without requiring programming knowledge.
- **NFR-06:** I will use readable text, consistent layouts, and clear result labels.

Reliability and Data Integrity

- **NFR-07:** I will handle unsupported files without crashing the application.
- **NFR-08:** I will validate records before saving them.
- **NFR-09:** I will use database keys and relationships to maintain consistent records.
- **NFR-10:** I will avoid treating missing information as a confirmed match.
- **NFR-11:** I will test the main processing functions with repeatable test cases.

Security and Privacy

- **NFR-12:** I will store project data locally by default.
- **NFR-13:** I will not require user accounts for the initial version.
- **NFR-14:** I will not store passwords, API keys, or other secrets in the repository.

Maintainability and Compatibility

- **NFR-15:** I will separate image processing, text matching, ranking, storage, and interface code into modules.
- **NFR-16:** I will record dependencies in a requirements file.
- **NFR-17:** I will provide setup and execution instructions in the README.
- **NFR-18:** I will use Git to track changes throughout development.
- **NFR-19:** I will target a normal laptop running a supported Python environment.
- **NFR-20:** I will document known limitations and the meaning of the matching scores.

5. System Architecture

I will use a modular Python architecture with Streamlit as the user interface and SQLite as the local database.

The application will receive page images, text, and metadata through Streamlit. Pandas will organize the metadata, while OpenCV and scikit-image will compare visual features. RapidFuzz will compare text, and NumPy will support numerical calculations.

I will combine the available evidence into matching scores and use NetworkX to represent candidate relationships. OR-Tools will help identify valid page sequences under defined ordering constraints. SQLite will store page records and analysis results. Streamlit will display ranked matches and explain the supporting evidence.

I will keep these components separate so I can test and improve them individually.

Architecture diagram: I will provide a separate diagram showing the flow from user input to processing modules, matching and ordering, SQLite storage, and the results interface.

6. Data Design

I will use SQLite because the project is local and does not require a large database server.

I plan to create the following tables:

- **Manuscripts:** Manuscript ID, title, source, and description.
- **Pages:** Page ID, manuscript ID, image path, page number, text, and metadata.
- **PageFeatures:** Feature ID, page ID, image properties, and extracted features.
- **CandidateMatches:** Match ID, two page IDs, similarity scores, and supporting evidence.
- **ProposedSequences:** Sequence ID, manuscript ID, proposed order, score, and evaluation status.

I will use primary keys to identify records and foreign keys to connect pages and analysis results. I will validate records before saving them.

My main data sources will be publicly accessible collections from the British Library and Digital Bodleian. I may also use suitable material from the Internet Archive. I will check access and reuse conditions before using the data.

For evaluation, I will use manuscripts with known page sequences and create test cases by separating pages or hiding selected metadata.

Data model: I will provide a separate diagram showing the tables and their relationships.

7. Interfaces and APIs

I will build the interface in Streamlit rather than developing a separate frontend and REST API.

The application will have four main areas:

- **Input:** I will upload page images and enter available metadata.
- **Review:** I will check the uploaded records and identify missing information.
- **Analysis:** I will start page comparison and sequence generation.

- **Results and Evaluation:** I will inspect ranked matches, evidence scores, proposed sequences, and test results.

The interface will communicate directly with internal Python modules. This approach will keep the design simple and suitable for the project schedule.

8. Technology Stack and Design Decisions

I will use Python because it supports all the main processing tasks in my project.

I selected Streamlit for the interface, Pandas for metadata, NumPy for numerical operations, OpenCV and scikit-image for image processing, RapidFuzz for text matching, NetworkX for candidate graphs, OR-Tools for ordering constraints, and SQLite for local storage.

I will use Git and GitHub to manage source code and record development progress. I will use a Python dependency file to make the environment easier to set up.

I considered a separate web frontend and a server-based database, but I believe they would add unnecessary complexity. I will use local tools unless the proof of concept shows a clear need for another approach.

9. Technical Proof of Concept

I will first test whether text and image comparison can identify useful candidate page matches.

I will select a small set of manuscript pages with a known order. I will compare available text using RapidFuzz and compare visual features using OpenCV or scikit-image. I will then calculate basic matching scores and display the strongest candidate relationships.

I will record the input data, scores, execution time, and observations in my repository. This proof of concept will help me check whether the selected libraries work with my data before I build the complete application.

I will report actual results after running the experiment. I will not claim that the matching method works accurately until I have tested it.

10. Testing and Evaluation Plan

I will use unit tests to check text similarity, image comparison, score calculation, and ordering constraints. I will use integration tests to check that the processing modules, database, and interface work together.

For the main evaluation, I will compare proposed sequences with known original orders. I will measure whether the correct next page appears among the top five candidates and whether the correct sequence appears among the highest-ranked results.

I will also test the target of processing 50 page images within 30 seconds. I will check how the application handles missing text, incomplete metadata, and unsupported files.

I will record the results and use them to identify weaknesses. This will allow me to evaluate the application using evidence rather than relying only on a successful demonstration.

11. Risks and Technical Challenges

My main risk is that historical images may be damaged, faded, or visually different. I will reduce this risk by choosing a manageable dataset and testing different page conditions.

Text and image similarity may also produce incorrect matches. I will combine different types of evidence and display the reasons behind each result.

The number of possible page sequences may increase quickly as the dataset grows. I will limit the initial dataset and reduce candidate relationships before applying ordering constraints.

Finally, I will manage the semester deadline by completing the proof of concept first and prioritizing the core matching workflow over optional features.

12. Development Schedule

- **October 19–25, 2026:** I will finalize the requirements, architecture, and data model.
- **October 26–November 1, 2026:** I will complete the text and image comparison proof of concept.
- **November 2–8, 2026:** I will implement metadata processing, scores, and candidate graphs.
- **November 9–15, 2026:** I will implement ordering constraints, storage, and the main interface.
- **November 16–22, 2026:** I will integrate the components and prepare the minimum viable application.
- **November 23–29, 2026:** I will complete functional, integration, and performance testing.
- **November 30–December 4, 2026:** I will improve the application and finish the documentation.
- **December 5–8, 2026:** I will prepare the final system, repository, evaluation results, and presentation.

13. Conclusion

I aim to develop a working application that helps researchers compare separated historical manuscript pages and explore possible page sequences. I will combine text, image, and metadata evidence with graph relationships and ordering constraints. I will test the application using known page sequences and clearly explain its limitations. My goal is to provide a useful research aid without claiming that every proposed arrangement is historically correct.

References

Bunke, H., & Riesen, K. (2011). Recent advances in graph-based pattern recognition with applications in document analysis. *Pattern Recognition, 44*(5), 1057–1067. DOI: https://doi.org/10.1016/j.patcog.2010.11.015

Diao, H., Zhang, Y., Ma, L., & Lu, H. (2021). Similarity reasoning and filtration for image-text matching. *Proceedings of the AAAI Conference on Artificial Intelligence, 35*(2), 1218–1226. DOI: https://doi.org/10.1609/aaai.v35i2.16209

Liu, L., Lu, Y., & Suen, C. Y. (2014). Near-duplicate document image matching: A graphical perspective. *Pattern Recognition, 47*(4), 1653–1663. DOI: https://doi.org/10.1016/j.patcog.2013.11.006

van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Neil, Y., Yu, T., & the scikit-image contributors. (2014). scikit-image: Image processing in Python. *PeerJ, 2*, e453. DOI: https://doi.org/10.7717/peerj.453

British Library. (n.d.). Digitised manuscripts. https://www.bl.uk/manuscripts/

Digital Bodleian. (n.d.). Digital Bodleian collections. https://digital.bodleian.ox.ac.uk/

Internet Archive. (n.d.). Internet Archive digital collections. https://archive.org/